

第 09 章 | 機器學習模型部署：從實驗到應用程式

學生版公開講義 | 請搭配本課程 Colab，依照段落逐步操作與自我檢查。

來源說明：課程參考《Python Machine Learning, 3rd Edition》；原始範例程式碼採 MIT License。本檔由恩恩統計家教整理為學生版學習摘要，不含原書 PDF 或掃描內容。

模型在 Notebook 中取得良好指標後，仍需包裝成穩定且回應時間可接受的服務。本章將模型串接到網頁介面與資料庫，並說明部署後的更新流程。

1. 本章學習路線與學習目標 (Chapter Positioning & Learning Objectives)

學習重點分析

在機器學習生命週期中，「部署 (Deployment)」是實現價值的最後一哩路。模型若無法嵌入網頁應用程式，其商業影響力將受限於開發者的單機環境。本章節是從「單機實驗」過渡到「多人連線」的思維轉換，這不僅是程式碼的搬運，更是對系統穩定性、環境一致性與資料持久化的工程挑戰。

學習目標清單

完成本章後，你將掌握以下五大核心工程技能：

- [] 模型序列化技術：利用 Pickle 凍結模型狀態，學會如何「去耦合」外部依賴（如 NLTK）。
- [] Flask 輕量化架構：建立網頁服務介面，理解微型框架在快速原型開發中的實務地位。
- [] 資料庫整合與中繼：掌握 SQLite 無伺服器架構，建立模型更新所需的「回饋迴路」。
- [] 生產環境部署實務：克服環境差異（Environment Parity），將服務推向公有雲。
- [] 線上學習機制 (Online Learning)：實作增量更新，確保模型具備適應真實資料分布變化的能力。

本章學習路線說明

本章扮演「演算法實作」與「系統工程」之間的橋樑。我們將解決模型如何「走出 Jupyter」並在公網環境下「持續存活」的問題。首要任務是處理模型的持久化，讓伺服器能瞬間喚醒訓練成果。

2. 模型記憶的保存：Pickle 序列化技術 (Model Serialization)

學習重點分析

在生產環境中，伺服器每次重啟時「重新訓練模型」是工程災難。訓練過程耗費巨大運算資源，因此我們必須實現模型持久化 (Persistence)。這不僅是為了省時，更是為了確保生產環境中使用的權重與實驗室完全一致。

核心觀念白話說明

我們可以將訓練好的模型比喻為一位「畢業的資深醫生」，他的診斷經驗（權重）是昂貴的財富。Pickle 序列化就像是將這位醫生的經驗完整下載到一個「記憶隨身碟」中。診所伺服器開啟時，只需插上隨身碟讀取（反序列化），醫生即可立即診治，無需重新修讀醫學院課程。

重要術語中英對照表

術語 (Traditional Chinese)	英文對照 (English)	說明
序列化	Serialization	將 Python 物件狀態轉換為可跨平台儲存的位元組碼 (Bytecode)。
反序列化	Deserialization	將位元組碼還原成記憶體中可執行的 Python 物件。
模型持久化	Model Persistence	確保模型狀態超越單次程序生命週期，實現檔案級儲存。
無狀態轉換器	Stateless Transformer	如 HashingVectorizer，純數學映射，不具備需要學習的狀態。

技術重點與實際意義

解耦依賴：本章特別強調除 `classifier.pkl` 外，還需將 NLTK 的 `stopwords.pkl` 一併序列化。實務意義：這能讓生產伺服器無需安裝龐大的 NLTK 語料庫即可運作，實現極簡部署。

記憶體效率：`HashingVectorizer` 不需要序列化，因為它是「無狀態」的。它不需像 `CountVectorizer` 在記憶體中建立字典，這意味著其記憶體足跡 (Memory Footprint) 是固定的，對於大規模併發請求的生產環境至關重要。

版本約束：使用 `protocol=4` 進行序列化可獲得最佳壓縮比，但這要求環境必須為 Python 3.4+。為求穩定，實務建議使用 Python 3.7 或更新版本。

⚠️ 資安警告：Pickle 執行風險 `pickle.load` 會直接在作業系統底層執行程式碼。載入不明來源的 `.pkl` 檔等同於授權駭客執行任意指令。專業建議：企業環境若需更高安全性，可評估 JSON (純資料) 或 Joblib (大型 NumPy 陣列最佳化) 作為替代方案。

3. 微型網頁架構：Flask 框架與專案骨架 (Flask Web Framework)

學習重點分析

為何資料科學家偏好 Flask 而非 Django？在 AI 工程中，我們需要的是「輕量、靈活且能快速將模型轉化為 API」的框架。Flask 的微型化架構能將工程開銷降至最低，讓團隊專注於演算法邏輯。

核心觀念白話說明

想像 Flask 是「餐廳外場經理」：

路由 (Routing)：決定客人輸入的網址該交給哪位廚師處理。

渲染 (Rendering)：將廚房做好的預測結果（變數）裝入名為 HTML 的信封中遞給使用者。

規矩與結構：生產級專案樹

嚴格的目錄結構是避免 `ModuleNotFoundError` 的第一防線：

```
movieclassifier/
├── app.py          # 餐廳經理：Flask 伺服器主程式
├── vectorizer.py   # 預處理邏輯：定義 tokenizer 與 HashingVectorizer
├── reviews.sqlite  # 資料中繼站：SQLite 資料庫檔案
├── pkl_objects/    # 儲藏室：存放序列化模型與停用字
│   ├── classifier.pkl
│   └── stopwords.pkl
├── static/         # 靜態資源：CSS 樣式表與圖檔
└── templates/      # 樣板間：Jinja2 HTML 範本 (務必拼對 s)
```

`@app.route` 裝飾器：將 URL 直接映射到 Python 函數。

GET vs POST：GET 用於請求顯示網頁；當接收影評文字時，必須使用 POST。POST 將資料封裝在訊息主體中，能處理大量文字並提供較佳的封裝性。

4. 輕量級資料中繼站：SQLite 資料庫 (Data Storage with SQLite)

學習重點分析

在開發初期，管理獨立資料庫伺服器會增加維運成本。SQLite 將資料庫存成單一檔案，不需另行管理伺服器，適合輕量應用程式收集回饋。

核心觀念白話說明

SQLite 在此扮演系統的「錯題紀錄本」。當使用者對預測結果給予糾正時，我們將真實資料存入這個單一檔案中。

程式碼實作：標準三部曲

```
import sqlite3
# 1. 連線：建立管道
conn = sqlite3.connect('reviews.sqlite')
c = conn.cursor()
# 2. 執行：推入資料
c.execute("INSERT INTO review_db (review, sentiment, date) VALUES (?, ?, DATETIME('now'))", (text, label))
# 3. 提交：確認寫入
conn.commit()
conn.close()
```

[!IMPORTANT] 深度警示：記得 `conn.commit()` 的實務意義等同於「編輯後存檔」。若漏掉此步，資料僅會留在揮發性記憶體中，當連線關閉時所有回饋將永久流失。

5. 雲端實戰：Colab 限制與 PythonAnywhere 部署 (Deployment Strategy)

學習重點分析

「本地開發」僅能解決個案需求。真正的「跨平台服務」必須部署至公有雲。這涉及解決環境對等性 (Environment Parity) 問題，確保雲端伺服器具備正確的 Python 版本與路徑設定。

Colab 操作陷阱：Localhost 的隱身衣

技術分析：在 Google Colab 執行 Flask 會導致儲存格進入死鎖，且因 Colab 運作於 Google 內網虛擬機，其啟動的 `127.0.0.1` 對你的本機瀏覽器是不可見的。因此，本章節必須在本機環境或雲端主機執行。

PythonAnywhere 部署指南

環境檢查：建議選擇 Python 3.7+ 環境，確保與 `protocol=4` 的 Pickle 檔相容。

檔案上傳：透過 Web 介面上傳整個專案目錄結構（確保 `vectorizer.py` 與 `app.py` 在同一層級）。

WSGI 設定：確保路徑指向你的 `app.py` 中的 `app` 物件。

Reload：每次更新模型或程式後，必須執行 "Reload"，否則伺服器會繼續執行記憶體中的舊版模型。

6. 模型更新：線上學習與系統防護 (Online Learning & Maintenance)

學習重點分析

「線上學習 (Online Learning)」能建立資料驅動的競爭優勢。透過使用者回饋持續修正模型，能讓預測器隨時間更貼近真實的機率分布，解決「概念漂移 (Concept Drift)」問題。

核心演算法：`partial_fit()`

與「完整重訓 (Full Retraining)」相比，`partial_fit` 是增量更新。它不需要重新掃描數百萬筆歷史資料，僅針對新的回饋 batch 進行梯度下降，極大地節省了生產伺服器的運算開銷。

工程防護機制 (Architectural Safeguards)

併發衝突 (Concurrency)：多個執行緒同時對 `.pkl` 寫入會導致檔案損毀。

備份 SOP：更新模型前，必須利用 `shutil` 建立帶有時間戳記的備份。

備份命名規範	意義
classifier_20231101-090000.pkl	2023年11月1日 09時整的穩定版本

7. 動手練習與自我檢查表 (Exercises & Checklist)

常見錯誤與排除手冊 (Troubleshooting)

錯誤現象	根本原因	解決方案
TemplateNotFound	templates/ 資料夾拼寫錯誤或位置不對。	確認資料夾名稱包含 's' 且與 app.py 同層級。
ModuleNotFoundError: No module named 'vectorizer'	vectorizer.py 不在 app.py 的同一路徑下。	檢查 Python 執行路徑或將檔案移動至 root。
sqlite3.OperationalError	資料庫檔案權限被鎖定或路徑為唯讀。	確認連線有正確 close()，並檢查雲端權限。

動手實作任務

結構化佈署：建立標準目錄結構並上傳所有 `.pkl` 檔案。

API 整合：實作 Flask 路由，能將影評字串轉換為特徵向量並呼叫分類器。

回饋迴路：完成資料庫 `INSERT` 邏輯，並確保 `commit()` 正確執行。

自我檢查清單

- ☐ 我是否已將 NLTK 停用字集 (`stopwords.pkl`) 打包以實現零依賴部署？
- ☐ 我能解釋為什麼 `HashingVectorizer` 不需要被序列化嗎？(提示：記憶體 footprint)
- ☐ 我的 `templates` 資料夾拼寫是否正確？
- ☐ 我是否在資料庫操作中正確使用了 `cursor` 與 `commit()`？
- ☐ 我是否理解在雲端更新模型前，先建立時間戳記備份的必要性？